PREDICTIVE ANALYTICS

Group Project Guidelines & Assignment

Academic Year 2025-26

1. Course Project Overview

The group project is a core component of the Predictive Analytics course, designed to provide students with real-world experience in building and deploying machine learning solutions. Each student will participate in three projects across three different groups, contributing meaningfully to every phase of the data science project life cycle.

This is not a box-ticking exercise. You are expected to approach every project with genuine curiosity and engineering rigour — think of it as a portfolio piece that will speak for you in interviews and on your professional profile. Do it with passion, not just to finish it.

Key Facts at a Glance

Projects Per Student	Each student completes 3 projects as part of 3 different groups.
Group Assignment	Groups are pre-assigned by the course instructor. Groups cannot be changed.
Project Selection	Teams may choose from the provided project list or propose their own topic.
Topic Restriction	A team cannot select a project topic already completed by any member in a prior project (Project 1 or Project 2).
Contribution	Every member must contribute to all stages of the project — no silent partners.

What "Contribution to All Stages" Means

The data science project life cycle is divided into the following stages. Each member must have demonstrable contributions in each:

1. **Stage 1:** Problem Definition & Literature Review
2. **Stage 2:** Data Collection & Data Understanding

3. **Stage 3:** Data Preprocessing & Cleaning
4. **Stage 4:** Exploratory Data Analysis (EDA)
5. **Stage 5:** Feature Engineering & Selection
6. **Stage 6:** Model Building & Training
7. **Stage 7:** Model Evaluation & Comparison
8. **Stage 8:** Model Interpretation & Explainability
9. **Stage 9:** Deployment (Streamlit or equivalent)
10. **Stage 10:** Documentation (README, PPT, GitHub)

How Contribution is Tracked

Your individual GitHub activity profile is the primary evidence of contribution. This includes:

- Git commit history — frequency, quality, and meaningful messages
- Branches created and merged
- Pull requests opened, reviewed, and commented on
- Issues raised and resolved
- Code reviews given to teammates

The instructor will review each member's GitHub activity profile at the time of submission. Submitting your teammate's work under your name or making trivial commits (e.g., editing a space) will not count as genuine contribution.

Marking Scheme

The project is worth 80% of the total course marks. The remaining 20% is assigned through other assessments (quizzes, viva, or exams as communicated separately). The 80% project mark is broken down as follows:

Assessment Component	Weight	Notes
GitHub Collaboration (commit history, branching, PR reviews, individual activity profile)	**30%**	All members must have meaningful commits
Deployment on Streamlit (or similar platform)	**15%**	Working live deployment link required
Data Science Life Cycle Coverage (all stages)	**15%**	Problem framing to model deployment
Project Difficulty Level	**10%**	Complexity and novelty considered
README File (detailed, with images and screenshots of deployment)	**10%**	Include architecture diagrams, results
PPT Presentation (in GitHub, with deployment screenshots)	**10%**	Professional slides with visual evidence
Individual Member Contribution (all stages)	**10%**	Each member must contribute to all phases

Note: The remaining 20% of course marks come from other assessments (presentation). The above 100% applies to the project component only.

Detailed Assessment Criteria

GitHub Collaboration (30%)

This is the most heavily weighted component because industry employers and research groups universally use GitHub activity as evidence of your capability. Your GitHub repository must clearly demonstrate:

- A meaningful commit history from every team member — not just one person committing all work.
- Use of branches: feature branches for different tasks (e.g., feature/eda, feature/modelling, feature/deployment).
- Pull Requests (PRs) with descriptions and code review comments — not just "LGTM".
- An Issues tracker used to assign tasks and track progress.
- Individual activity profiles must be submitted alongside the final deliverable — screenshot or export of each member's contribution graph.

Deployment on Streamlit (15%)

Your model must be deployed on a publicly accessible platform. Streamlit Community Cloud is the recommended and easiest option. The deployment must:

- Be live and accessible via a shareable URL at the time of submission.
- Accept relevant inputs from a user (manually entered or via file upload).
- Display model predictions with appropriate visualizations (confidence scores, class probabilities, feature importance charts, etc.).
- Handle invalid or edge-case inputs gracefully without crashing.
- Include screenshots and a link to the live app in both the README and the PPT.

Data Science Life Cycle Coverage (15%)

All ten stages of the life cycle listed in Section 3.1 must be visibly and thoroughly addressed in your code, notebooks, and documentation. Skipping stages (e.g., going from raw data directly to modelling without EDA or feature engineering) will result in significant deductions. Your notebooks should be clean, well-commented, and reproducible.

Project Difficulty Level (10%)

Projects are implicitly tiered by difficulty. Teams that take on more challenging datasets, implement more sophisticated algorithms, or address harder real-world constraints (e.g., class imbalance, multi-modal data, interpretability requirements) will be rewarded. Do not pick the easiest route — reviewers will notice.

README File (10%)

Your README.md file is the front door of your project. It must include:

- Project title, team members, and course details.
- Problem statement and motivation.
- Dataset description (source, size, features, class distribution).
- Methodology overview — all life cycle stages covered.
- Results summary with evaluation metrics and model comparisons.
- Screenshots and images of your deployed application.
- Instructions for setting up and running the project locally.
- Link to the live Streamlit deployment.

PPT Presentation (10%)

A professional PowerPoint presentation must be uploaded to your GitHub repository. It should cover:

- Problem statement and dataset overview.
- Methodology and pipeline diagram.
- Key results, charts, and model comparisons.
- Screenshots of the deployed application.
- Conclusions, limitations, and future work.

Individual Member Contribution (10%)

Beyond GitHub activity, each member will be assessed on their demonstrated involvement in all project stages. This may include a brief individual viva or contribution statement. Unequal contribution identified by the instructor may result in individual mark adjustments irrespective of the team's overall performance.

Submission Requirements

GitHub Repository

Submit the URL of your GitHub repository. The repository must be public (or shared with the instructor). It must contain:

- All source code and Jupyter notebooks.
- README.md with all required sections.
- The PPT file (PDF or .pptx).
- A folder named /individual_profiles/ with exported GitHub activity screenshots for each member.
- A requirements.txt or environment.yml for reproducibility.

Deployment Link

The live Streamlit (or equivalent) deployment URL must be included in the README and submitted separately to the instructor via the course portal.

Project Topics

The following 40 projects are approved for selection. Teams may also propose their own topic subject to instructor approval. Remember: no team member may work on the same topic they completed in a previous project.

Project #1	**Breast Cancer Tumor Classification Using Clinical Features** Build a classification model to distinguish malignant from benign breast tumors using the Wisconsin Breast Cancer Dataset. Apply Logistic Regression, SVM, and Random Forest. Features include cell nucleus measurements. Evaluate with accuracy, precision, recall, F1-score, and ROC-AUC. Perform feature importance analysis and cross-validation.
Project #2	**Credit Card Fraud Detection System** Develop a binary classification system to identify fraudulent credit card transactions from the Kaggle credit card fraud dataset. Address severe class imbalance using SMOTE and cost-sensitive learning. Compare Decision Trees, XGBoost, and Neural Networks. Focus on minimizing false negatives using precision-recall curves and cost-benefit analysis.
Project #3	**Email Spam Classification Using NLP and Machine Learning** Create a spam detection classifier using the SpamAssassin or Enron email dataset. Apply TF-IDF and word embeddings for feature extraction. Train Naive Bayes, SVM, and LSTM models. Evaluate performance using confusion matrix and F1-score. Implement a preprocessing pipeline for email cleaning, tokenization, and stop-word removal.
Project #4	**Heart Disease Prediction Using Patient Health Data** Predict the presence or absence of heart disease using the Cleveland Heart Disease Dataset with 13 clinical attributes (age, cholesterol, ECG results, etc.). Apply KNN, Logistic Regression, and Gradient Boosting. Perform feature selection using mutual information and chi-square tests. Visualize decision boundaries and feature correlations.
Project #5	**Customer Churn Prediction for Telecom Companies** Classify telecom customers as churners or non-churners using usage behavior and demographics. Use the Telco Customer Churn dataset. Apply SMOTE for class

	imbalance, train XGBoost and Random Forest models, and perform SHAP analysis for model explainability. Output actionable customer retention insights backed by model predictions.
Project #6	**Diabetic Retinopathy Detection from Fundus Images** Classify retinal fundus images into 5 severity grades of diabetic retinopathy using the Kaggle DR dataset. Apply transfer learning with ResNet-50 or EfficientNet. Handle class imbalance with weighted loss functions. Use Grad-CAM for visual explanations of model decisions. Evaluate with quadratic weighted kappa score.
Project #7	**Sentiment Analysis of Product Reviews** Classify customer reviews (positive, negative, neutral) from Amazon or Yelp datasets. Use BERT or DistilBERT for feature extraction and fine-tuning. Compare with traditional ML methods (SVM with TF-IDF). Analyze model performance across product categories. Build a deployment-ready pipeline using Flask or FastAPI.
Project #8	**Diabetic Retinopathy Severity Classification from Fundus Images** Classify retinal fundus images into 5 grades of diabetic retinopathy (No DR, Mild, Moderate, Severe, Proliferative). Extract handcrafted features: HOG for texture, color histograms (RGB/HSV channels), and GLCM for lesion texture. Apply PCA for dimensionality reduction. Train SVM with RBF kernel and Random Forest classifiers. Evaluate using quadratic weighted kappa, per-class F1-score, and confusion matrix. Compare feature combinations to identify the most discriminative descriptor.
Project #9	**Sentiment Analysis of Product Reviews Using TF-IDF and ML Classifiers** Classify customer reviews (positive, negative, neutral) from Amazon or Yelp datasets without deep learning. Extract linguistic features: TF-IDF unigrams and bigrams, sentence length, punctuation density, average word length, and presence of negation words. Apply chi-square and mutual information for feature selection. Train and compare Logistic Regression, Multinomial Naive Bayes, Linear SVM, and Gradient Boosting. Report macro F1-score and per-class precision-recall. Analyze top discriminating features per sentiment class.
Project #10	**Kerala Flower & Plant Disease Classification Using Color and Texture Features** Classify common plant diseases visible on crop leaves (healthy, bacterial spot, early blight, late blight, etc.) using the PlantVillage dataset. Extract handcrafted image features: color histograms in HSV space, LBP (Local Binary Patterns) for texture, and Hu moments for shape. Concatenate feature vectors and apply feature selection via mutual information. Train SVM, KNN, and Random Forest. Compare individual vs. combined feature sets. Evaluate per-class accuracy and F1-score with 5-fold cross-validation.

Project #11	**Loan Default Risk Classification for Banking** Predict whether a loan applicant will default using the Home Credit or LendingClub dataset. Features include income, credit history, and loan amount. Apply class-weighted Logistic Regression, Random Forest, and LightGBM. Optimize for recall to minimize risky approvals. Use SHAP for regulatory-compliant model explainability.
Project #12	**Human Activity Recognition from Smartphone Sensors** Classify physical activities (walking, sitting, standing, running, stair climbing) from accelerometer and gyroscope data in the UCI HAR Dataset. Extract time-domain and frequency-domain features. Train SVM, Random Forest, and LSTM models. Compare handcrafted vs. deep learning features. Evaluate on unseen subjects for generalization.
Project #13	**Fake News Detection Using NLP Classifiers** Identify fake vs. real news articles using the LIAR or FakeNewsNet dataset. Apply linguistic features (readability, punctuation patterns), TF-IDF, and BERT embeddings. Train Logistic Regression, Gradient Boosting, and transformer models. Analyze linguistic patterns unique to fake news. Evaluate cross-domain generalization capability.
Project #14	**Skin Lesion Classification for Dermatology (HAM10000)** Classify skin lesion images into 7 categories (melanoma, basal cell carcinoma, etc.) using the HAM10000 dataset. Address extreme class imbalance with oversampling and augmentation. Apply EfficientNet-B4 with transfer learning. Report class-wise sensitivity and specificity. Compare performance against dermatologist-level benchmarks.
Project #15	**Water Quality Classification for Safe Drinking Prediction** Classify water samples as potable or non-potable using physicochemical properties (pH, hardness, chloramines, sulfate, etc.) from the Water Potability dataset. Apply Random Forest, SVM, and XGBoost. Handle missing values, perform feature engineering, and use SMOTE. Interpret results for environmental monitoring applications.
Project #16	**Skin Lesion Type Classification Using Texture and Color Features with SVM** Classify skin lesion images into 7 categories (melanoma, basal cell carcinoma, etc.) using the HAM10000 dataset without deep learning. Extract GLCM texture features (contrast, correlation, energy, homogeneity), HSV color histograms, and asymmetry/border irregularity shape descriptors mimicking clinical ABCD criteria. Apply SMOTE for class imbalance. Train SVM with RBF kernel, Random Forest,

and XGBoost. Report class-wise sensitivity and specificity. Compare feature groups to identify clinically relevant discriminators.

Project #17

Malware Detection and Classification Using System Logs

Classify Windows PE files as benign or malicious (ransomware, trojans, worms) using the EMBER or CIC-MalMem dataset. Extract static features (byte n-grams, API calls, PE headers). Apply Random Forest, LightGBM, and MLP. Evaluate detection rate and false positives. Discuss adversarial robustness and model evasion techniques.

Project #18

Employee Attrition Prediction Using HR Analytics

Predict employee attrition (yes/no) using the IBM HR Analytics dataset with features like job satisfaction, overtime, years at company, and department. Apply Logistic Regression, Decision Tree, and XGBoost. Perform chi-square and ANOVA feature selection. Use SHAP for HR-interpretable explanations to guide retention strategies.

Project #19

Traffic Sign Recognition for Autonomous Vehicles

Classify 43 categories of traffic signs from the German Traffic Sign Recognition Benchmark (GTSRB). Build CNN models with batch normalization and dropout. Apply histogram equalization for preprocessing. Evaluate accuracy under varying lighting conditions. Analyze failure cases and discuss safety implications for self-driving systems.

Project #20

Crop Recommendation System Using Soil and Climate Data

Recommend the most suitable crop for farming based on soil nutrients (N, P, K), temperature, humidity, pH, and rainfall using the Crop Recommendation dataset. Train multi-class classifiers (Random Forest, KNN, Decision Tree). Visualize decision regions and feature importances. Build an interactive web app for farmers using Streamlit.

Project #21

Traffic Sign Recognition Using HOG Features and SVM Classifier

Classify 43 traffic sign categories from the GTSRB dataset using handcrafted features instead of deep learning. Apply histogram equalization for illumination normalization. Extract HOG features at multiple scales, combined with color channel histograms (RGB, HSV). Use PCA to reduce HOG feature dimensions. Train multi-class SVM (one-vs-one), KNN, and Random Forest. Evaluate accuracy per sign category. Analyze failure cases and discuss how lighting and occlusion affect HOG descriptor quality.

Project #22

Multi-Class Classification of Galaxy Morphologies

Classify galaxy images into elliptical, spiral, and irregular types using the Galaxy Zoo dataset. Apply CNN architectures (ResNet-18, VGG-16) with transfer learning.

Use data augmentation for rotation-invariant training. Evaluate per-class metrics and visualize feature maps. Compare with traditional HOG+SVM approach.

Project #23

Student Performance Prediction and Early Intervention

Predict whether students will pass or fail based on demographics, attendance, study time, and prior grades using the UCI Student Performance dataset. Train Decision Tree, Logistic Regression, and Gradient Boosting classifiers. Identify at-risk students early. Use LIME for per-student explanations to guide counselor interventions.

Project #24

Intrusion Detection in Network Traffic Using ML

Classify network traffic as normal or as specific attack types (DoS, probe, R2L, U2R) using the NSL-KDD or CIC-IDS2017 dataset. Apply feature selection to reduce dimensionality. Train Random Forest, XGBoost, and MLP models. Evaluate detection rates per attack class. Discuss real-time deployment challenges and latency constraints.

Project #25

Wildfire Risk Classification from Satellite and Weather Data

Classify regions into wildfire risk levels (low, medium, high) using meteorological features (temperature, wind speed, humidity, drought index) and satellite-derived vegetation indices. Use the Algerian Forest Fire or NASA FIRMS dataset. Apply SVM, Random Forest, and XGBoost. Visualize risk maps and evaluate spatial prediction accuracy.

Project #26

Mental Health Status Classification from Social Media Text

Classify Reddit or Twitter posts into mental health categories (depression, anxiety, PTSD, normal) using NLP techniques. Apply BERT fine-tuning and compare with TF-IDF + SVM baselines. Address privacy and ethical considerations. Evaluate sensitivity and specificity. Discuss potential for early mental health intervention tools.

Project #27

Quality Control: Defect Detection in Manufacturing Images

Classify product images as defective or non-defective using the MVTec Anomaly Detection dataset. Apply CNN-based classifiers and anomaly detection methods (autoencoders). Use Grad-CAM to localize defect regions. Evaluate precision and recall trade-offs. Discuss integration into industrial IoT pipelines for real-time inspection.

Project #28

Multi-Label Scene Classification in Satellite Imagery

Classify satellite image patches into multiple land-use categories (forest, agriculture, water, urban) using the EuroSAT or BigEarthNet dataset. Apply multi-label ResNet classifiers with sigmoid outputs. Use binary cross-entropy loss. Evaluate with Hamming loss and subset accuracy. Discuss applications in environmental monitoring and urban planning.

Project #29

Manufacturing Defect Detection Using Texture Features and ML Classifiers

Classify product images as defective or non-defective using the MVTec or NEU surface defect dataset without deep learning or autoencoders. Extract LBP, GLCM texture features (energy, contrast, entropy), and Gabor filter responses at multiple orientations and scales. Concatenate features and apply feature selection. Train SVM, Random Forest, and Gradient Boosting. Use ROC-AUC and precision-recall curves to select operating threshold. Analyze which texture features best capture surface anomalies per defect type.

Project #30

Land Use Classification from Satellite Imagery Using GLCM and ML Models

Classify satellite image patches into land-use categories (forest, agriculture, water, urban, bare land) using the EuroSAT or UC Merced Land Use dataset without deep learning. Extract GLCM texture features per spectral band, color histograms, and spatial frequency features via Gabor filters. Apply PCA for dimensionality reduction. Train SVM, Random Forest, and KNN classifiers. Evaluate Hamming loss and per-class F1-score. Discuss how handcrafted spatial features compare to learned representations for remote sensing classification.

Project #31

Kerala Flower Species Classification Using Color Histograms and SVM

Classify native Kerala flowers (Kanikonna/Golden Shower, Neelakurinji, Lotus, Hibiscus, Ixora, Jasmine, etc.) from collected or curated flower images. Extract handcrafted image features: HSV and RGB color histograms capturing petal color distribution, petal shape descriptors using Hu moments, and LBP for texture of petals and leaves. Combine features into a single descriptor vector. Apply feature selection using mutual information. Train multi-class SVM (one-vs-one), KNN, and Random Forest classifiers. Evaluate per-species accuracy, macro F1-score, and confusion matrix. Analyze which feature types (color vs. texture vs. shape) contribute most to correct classification.

Project #32

Handwritten Malayalam Alphabet Classification Using HOG and SVM

Classify handwritten Malayalam alphabet characters (vowels and consonants — 56 classes) from a self-collected or public dataset. Preprocess images with binarization (Otsu thresholding) and morphological normalization. Extract HOG features (block size 8x8, 9 orientation bins) and Zernike moments for shape. Concatenate descriptors into a fixed-length feature vector. Train multi-class SVM with RBF kernel, KNN, and Random Forest. Evaluate per-class and macro accuracy using 5-fold cross-validation. Analyze common misclassification pairs (visually similar characters) and discuss preprocessing impact on feature quality.

Project #33

Handwritten Digit Recognition Using HOG, Pixel Intensity and SVM

Classify handwritten digits (0-9) from the MNIST dataset using classical ML with manual feature extraction — no neural networks. Extract HOG features (captures stroke direction), raw pixel intensity vectors (28x28 flattened), and Zernike moment

shape descriptors. Apply PCA to reduce dimensionality to 50-100 components. Train and compare SVM (RBF kernel), KNN, Logistic Regression, and Random Forest. Report per-digit accuracy and confusion matrix. Investigate effect of different feature combinations and PCA components on classification accuracy. Discuss robustness to rotation and thickness variations.

Project #34

Mathematical Operator Symbol Classification from Handwritten Images

Classify handwritten mathematical operators (+, -, x, /, =, etc.) from a collected or synthesized dataset. Preprocess with binarization, skeletonization, and size normalization to 32x32 pixels. Extract HOG features for stroke orientation, pixel run-length features for stroke structure, and projection profiles (horizontal and vertical) as shape descriptors. Train multi-class SVM, KNN, and Decision Tree. Apply data augmentation (slight rotation, scaling) to increase robustness. Evaluate with macro F1-score and confusion matrix. Identify visually confusable pairs and analyze misclassification causes.

Project #35

Chess Piece Identification from Top-Down Board Images Using HOG and SVM

Classify chess pieces (King, Queen, Rook, Bishop, Knight, Pawn — both white and black — 12 classes) from segmented top-down chessboard square images. Preprocess by extracting individual squares and applying illumination normalization. Extract HOG features for shape contours, aspect ratio and area of the piece silhouette, and Hu moments for rotation-invariant shape description. Use PCA for dimensionality reduction. Train multi-class SVM (one-vs-one), KNN, and Random Forest classifiers. Evaluate per-piece classification accuracy and macro F1. Analyze how lighting variation and piece design differences affect HOG feature stability.

Project #36

Dice Face Number Recognition from Images Using Dot-Counting Features and SVM

Classify the face-up number of a standard six-sided dice (1-6) from photographed dice images using classical feature engineering without deep learning. Preprocess by segmenting the dice face via contour detection and applying perspective correction. Extract dot-counting features: Blob detection to count circular pips, spatial distribution of dots (centroid coordinates), and HOG features on the dice face region. Additionally use Hu moments for spatial arrangement. Train SVM, Decision Tree, and KNN classifiers. Evaluate accuracy across varied lighting and background conditions. Extend to classify tilted dice for robustness testing.

Project #37

Bird Sound Classification: Crow vs. Peacock vs. Other Birds Using MFCC and SVM

Classify bird vocalizations (crow, peacock, cuckoo, kingfisher, and background noise) from audio recordings using classical ML with audio feature extraction — no deep learning. Extract MFCC (Mel-Frequency Cepstral Coefficients — 13-40 coefficients), spectral centroid, spectral rolloff, zero crossing rate, and chroma features using Librosa. Compute statistical aggregates (mean, std, min, max) per feature over the audio clip. Train SVM with RBF kernel, KNN, and Random Forest.

	Evaluate with per-class F1-score and confusion matrix. Analyze which acoustic features best discriminate between bird species and background noise.

Project #38	**Speech vs. Music vs. Noise Audio Signal Classification Using MFCC and SVM** Classify audio signal types (speech, music, environmental noise, silence) using signal-level feature extraction without deep learning. Extract MFCC features (20 coefficients with delta and delta-delta), short-time energy, spectral flux, spectral centroid, spectral bandwidth, and zero crossing rate using Librosa. Compute statistical features (mean, variance, skewness) over fixed-length frames. Train SVM, KNN, and Gradient Boosting classifiers. Use the MUSAN or ESC-50 dataset. Evaluate classification accuracy, macro F1-score, and per-class precision-recall. Analyze feature importance and discuss robustness to SNR variations and recording quality.

Project #39	**Language Identification from Short Audio Clips Using MFCC and ML** Classify spoken language (Malayalam, Tamil, Hindi, English, Kannada) from 3-10 second audio clips using acoustic feature extraction and ML classifiers — no deep learning. Extract MFCC (39 coefficients with delta), prosodic features (pitch, energy, speaking rate), spectral features (formant frequencies F1-F3, spectral centroid), and phonotactic features (vowel/consonant ratio). Aggregate per clip with mean and covariance. Train SVM, Random Forest, and Logistic Regression. Evaluate macro F1-score with 5-fold cross-validation. Analyze which features capture phonological differences between Dravidian and Indo-Aryan languages.

Project #40	**Coin Denomination Classification from Images Using Shape and Texture Features** Classify Indian coin denominations (1, 2, 5, 10, 20 rupees) from photographed coin images using classical ML — no deep learning. Preprocess by segmenting the coin using circular Hough transform. Extract features: coin diameter ratio (normalized), edge density using Canny edge count, LBP texture for surface design patterns, HOG features for edge orientation of coin reliefs, and HSV color histogram for metal color differentiation. Train SVM, KNN, and Random Forest classifiers. Evaluate per-denomination accuracy and confusion matrix. Test robustness across worn, new, and differently-lit coin images.

Good luck — and remember: do it with passion, not just to finish it.